

OOP_Notebooks_ChatGPT

May 21, 2026

1 OOP Notebooks

ChatGPT-5.5 Plus Prompt:

Yes, lay out that 10 notebook sequence

I'd structure it as a mini course where each notebook introduces **one new OOP idea** and ends with a meaningful engineering-style exercise. Since you already know Python, I'd avoid toy examples like "Dog" and "Car" and stay close to signals, systems, instrumentation, and simulation.

2 Notebook 1 — Objects as State Containers

File: 01_objects_and_state.ipynb

2.0.1 Main idea

Objects store **state** + **behavior**.

Build:

Signal

State:

`samples`

`fs`

`name`

Methods:

`duration()`

`energy()`

`mean()`

`plot()`

Exercises:

1. Add RMS calculation
2. Add peak detection
3. Add signal normalization
4. Compare two signals

Reflection:

Which variables belong permanently to the signal?

Deliverable:

```
sig = Signal(x, 1000)
```

```
sig.energy()
```

```
sig.plot()
```

3 Notebook 2 — Responsibility and Encapsulation

File: 02_encapsulation.ipynb

3.0.1 Main idea

Protect object integrity.

Build:

```
class Signal:
```

Add validation:

```
fs > 0
```

```
samples not empty
```

Introduce:

```
@property
```

```
setter
```

```
private variables
```

Example:

```
sig.fs = -100
```

Should produce:

```
ValueError
```

Exercises:

1. Prevent invalid sample rate
2. Prevent NaNs
3. Add readonly duration property
4. Log state changes

Reflection:

What should users be allowed to change?

4 Notebook 3 — Objects Interacting

File: 03_object_collaboration.ipynb

Create chain:

Source
Encoder
Channel
Receiver

Pipeline:

```
src.generate()
```

```
enc.encode()
```

```
ch.transmit()
```

```
rx.detect()
```

Exercises:

Build:

- Binary source
- Noise channel
- Error counter

Questions:

- Who owns BER?
- Who owns noise variance?
- Should source know channel?

Goal:

Learn **separation of responsibility**.

5 Notebook 4 — Composition vs Inheritance

File: 04_composition_vs_inheritance.ipynb

Start wrong:

Signal
 BPSKSignal
 QPSKSignal
 OFDMSignal

Then redesign:

Signal

Modulator

Channel

Composition:

```
sig = Signal(bits)
```

```
mod = Modulator("BPSK")
```

```
wave = mod.modulate(sig)
```

Exercises:

1. Compare both architectures
2. Add QPSK
3. Add ASK
4. Measure complexity

Reflection:

When is “is-a” wrong?

6 Notebook 5 — Simulation Framework

File: 05_simulation_engine.ipynb

Create:

Simulation

Contains:

```
source
encoder
channel
receiver
metrics
```

Run:

```
sim.run()
```

Add:

```
reset()
step()
summary()
```

Exercises:

Run BER experiments.

Store:

`EbN0`

`BER`

`iterations`

Goal:

Learn orchestration objects.

7 Notebook 6 — Polymorphism

File: `06_polymorphism.ipynb`

Create interface style behavior:

`Channel`

Derived:

`AWGNChannel`

`FadingChannel`

`InterferenceChannel`

Same call:

`channel.transmit(x)`

Swap:

`sim.channel = AWGNChannel()`

`sim.channel = FadingChannel()`

Exercises:

1. Add impulsive noise
2. Add burst errors
3. Compare outputs

Reflection:

Why does identical API matter?

8 Notebook 7 — Instrument Modeling

File: `07_virtual_instruments.ipynb`

Build:

Oscilloscope
SpectrumAnalyzer
SignalGenerator

Interactions:

`gen.generate()`

`scope.measure()`

`spec.compute_fft()`

Exercises:

Add:

- trigger level
- autoscale
- peak hold
- bandwidth limits

Goal:

Objects with persistent state.

9 Notebook 8 — Dataclasses and Cleaner Design

File: 08_dataclasses.ipynb

Introduce:

```
from dataclasses import dataclass
```

Example:

```
@dataclass
class ChannelConfig:
```

```
    snr: float
```

```
    fading: bool
```

```
    seed: int
```

Exercises:

Convert:

SignalConfig

ReceiverConfig

SimulationConfig

Reflection:

When should classes contain data only?

10 Notebook 9 — Design Patterns (Practical)

File: 09_patterns.ipynb

Pattern 1:

Factory

```
factory.make_channel(  
    "awgn"  
)
```

Pattern 2:

Strategy

```
decoder = Viterbi()
```

```
decoder = MLDecoder()
```

Pattern 3:

Observer

```
scope.subscribe(signal)
```

Exercises:

Build channel factory.

Add decoder switching.

Goal:

See reusable architectures.

11 Notebook 10 — Capstone Project

File: 10_comm_system_project.ipynb

Build complete simulator:

BitSource

ConvolutionalEncoder

Interleaver

BPSKModulator

AWGNChannel

ViterbiDecoder

BERCounter

Simulation

Run:

```
sim = Simulation()
```

```
sim.run()
```

```
sim.plot_ber()
```

Add:

```
save_results()
```

```
export_csv()
```

```
parameter_sweep()
```

Final challenge:

Create:

MonteCarloRunner

for:

```
EbN0 = np.arange(  
    0,  
    10,  
    1  
)
```

Generate BER curves.

12 Recommended notebook workflow

Every notebook should start with:

```
# Object map
```

Signal

State:

- samples
- fs
- name

Behavior:

- energy()
- plot()
- fft()

Then:

Responsibilities:

Signal owns:

samples
sampling rate

Signal does NOT own:

channel noise
BER
decoder state

Then:

Design questions

1. What persists?
2. Who owns it?
3. What should remain hidden?
4. Which objects collaborate?

13 Difficulty progression

Notebook	Topic	Difficulty
1	Objects/state	Easy
2	Encapsulation	Easy
3	Collaboration	Medium
4	Composition	Medium
5	Framework design	Medium
6	Polymorphism	Medium–Hard
7	Instrument models	Hard
8	Dataclasses	Medium
9	Patterns	Hard
10	Full simulator	Advanced

Notebook	Topic	Difficulty
----------	-------	------------

If you complete all ten, you will move from “**I know Python syntax**” to “**I can architect medium-size OOP systems.**”

[]: